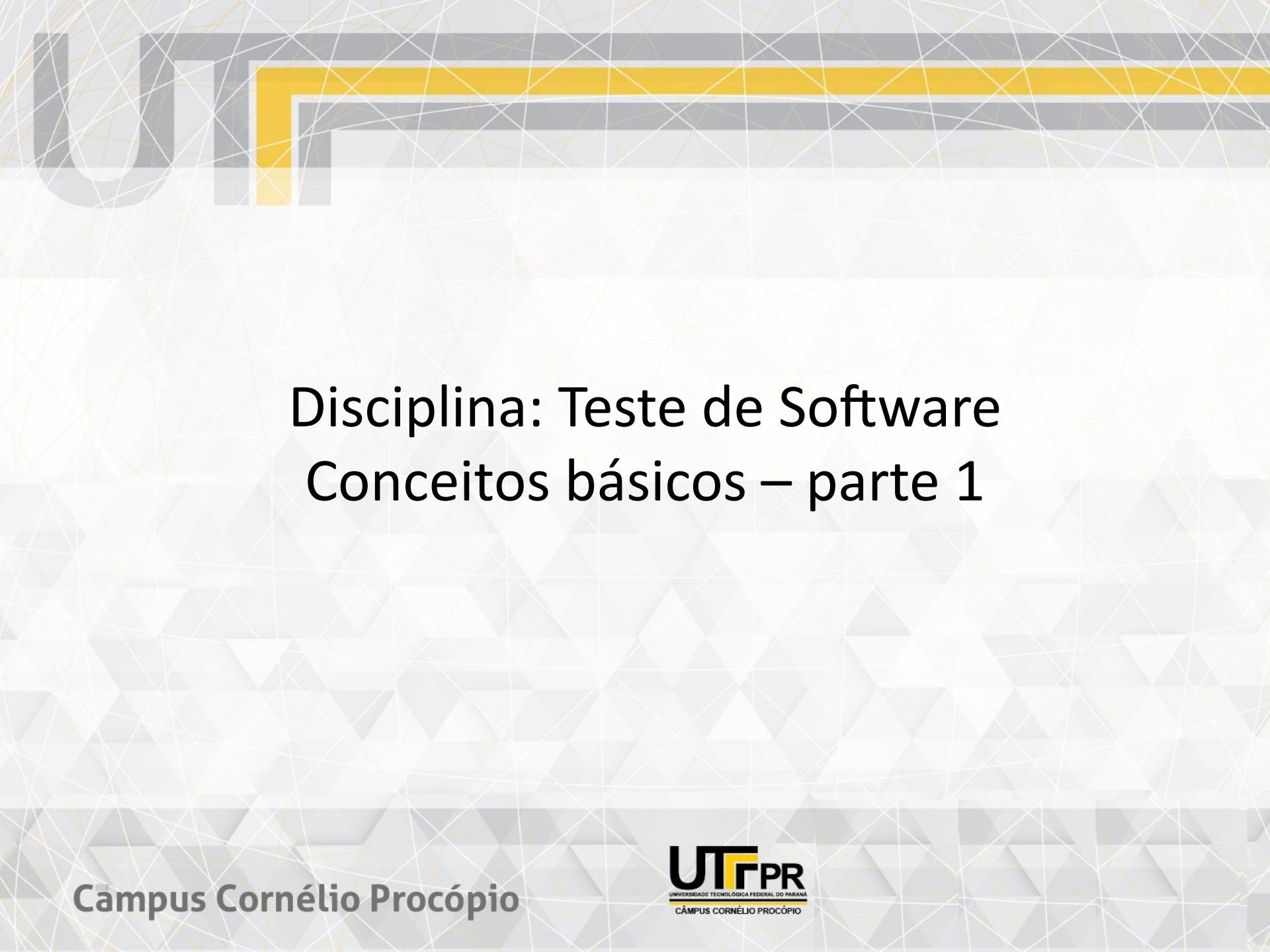


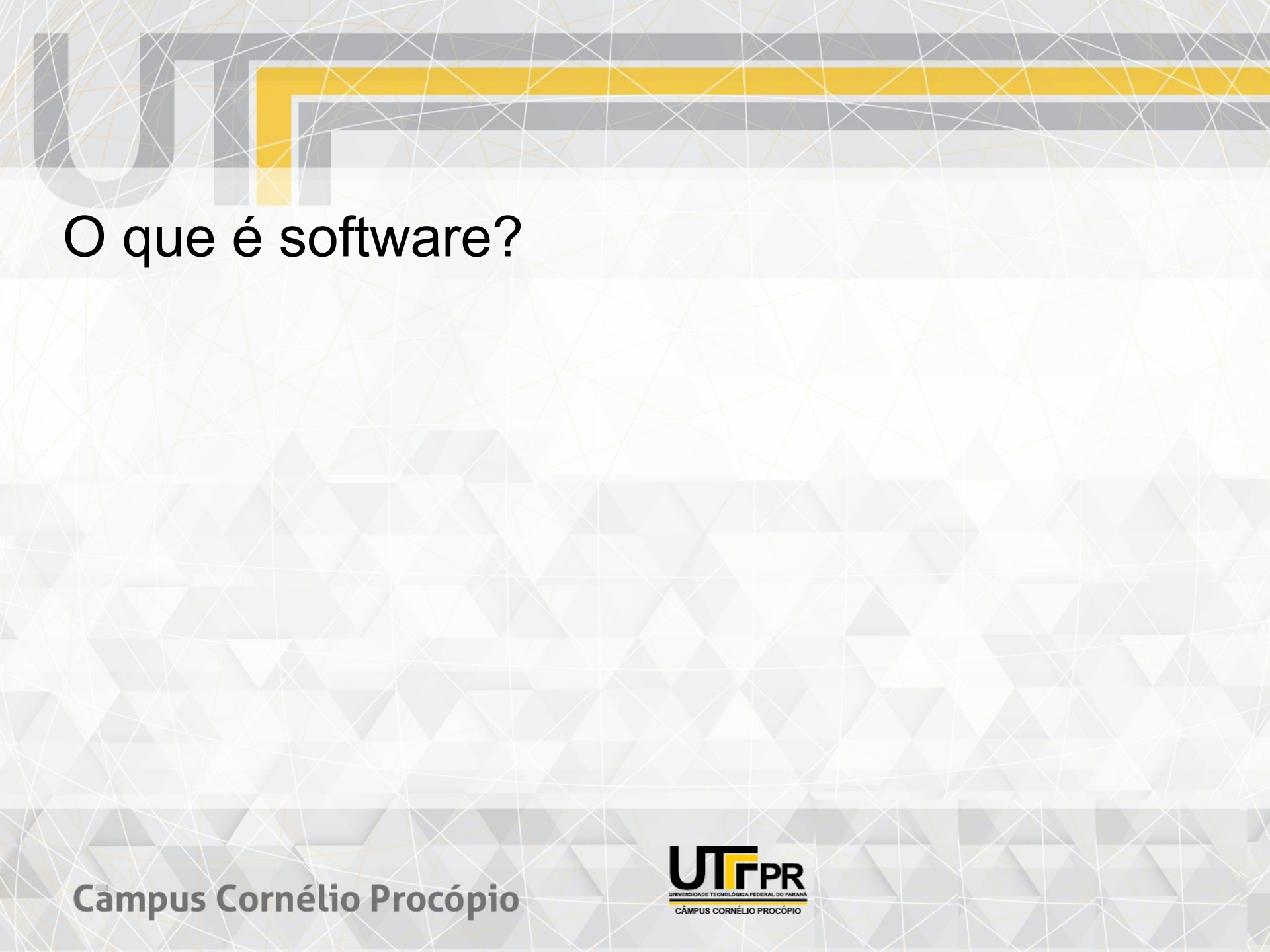


Disciplina: Teste de Software

Conceitos básicos – parte 1



Vamos começar



O que é software?

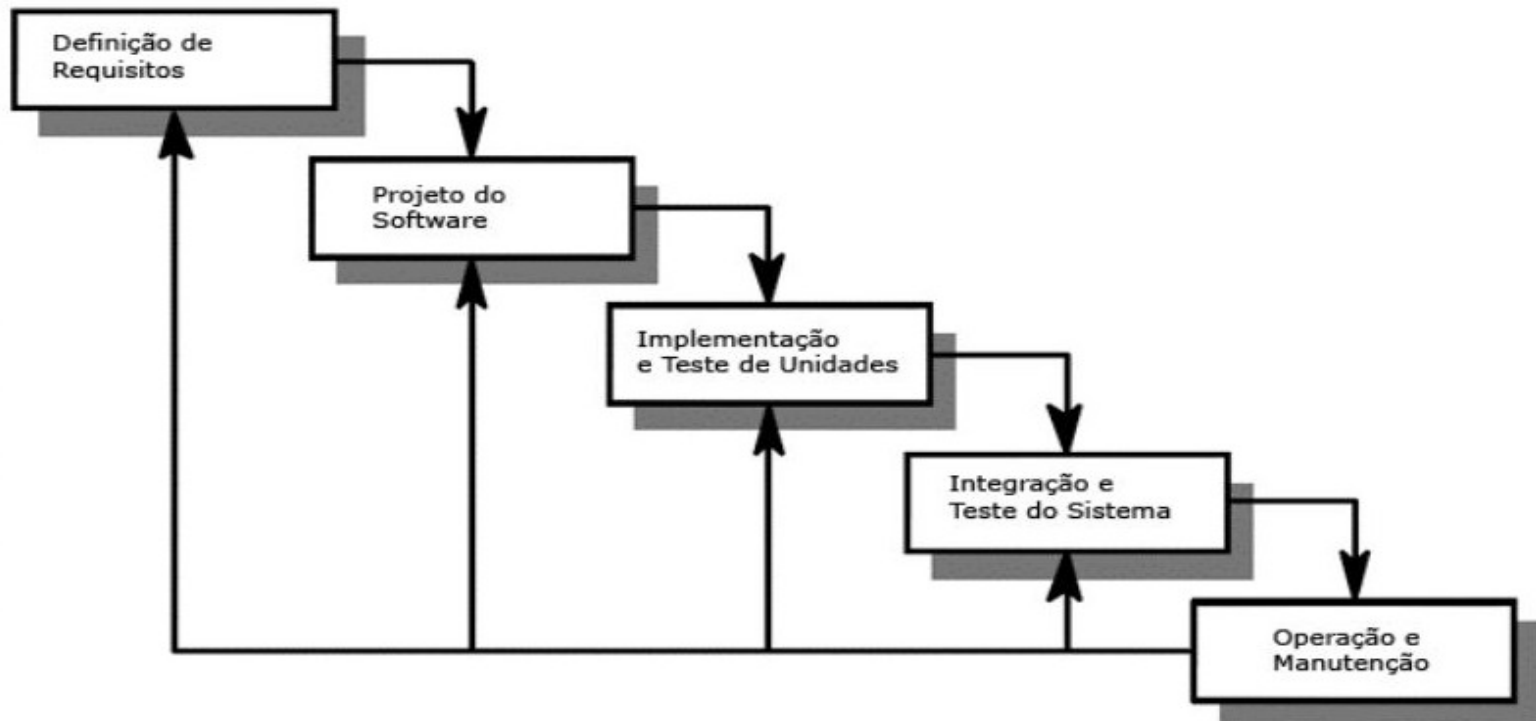
O que é software?

- Software é um programa de computador e documentação associada. Produtos de software podem ser desenvolvidos para um cliente específico ou para o mercado em geral (SOMMERVILLE, 2011)

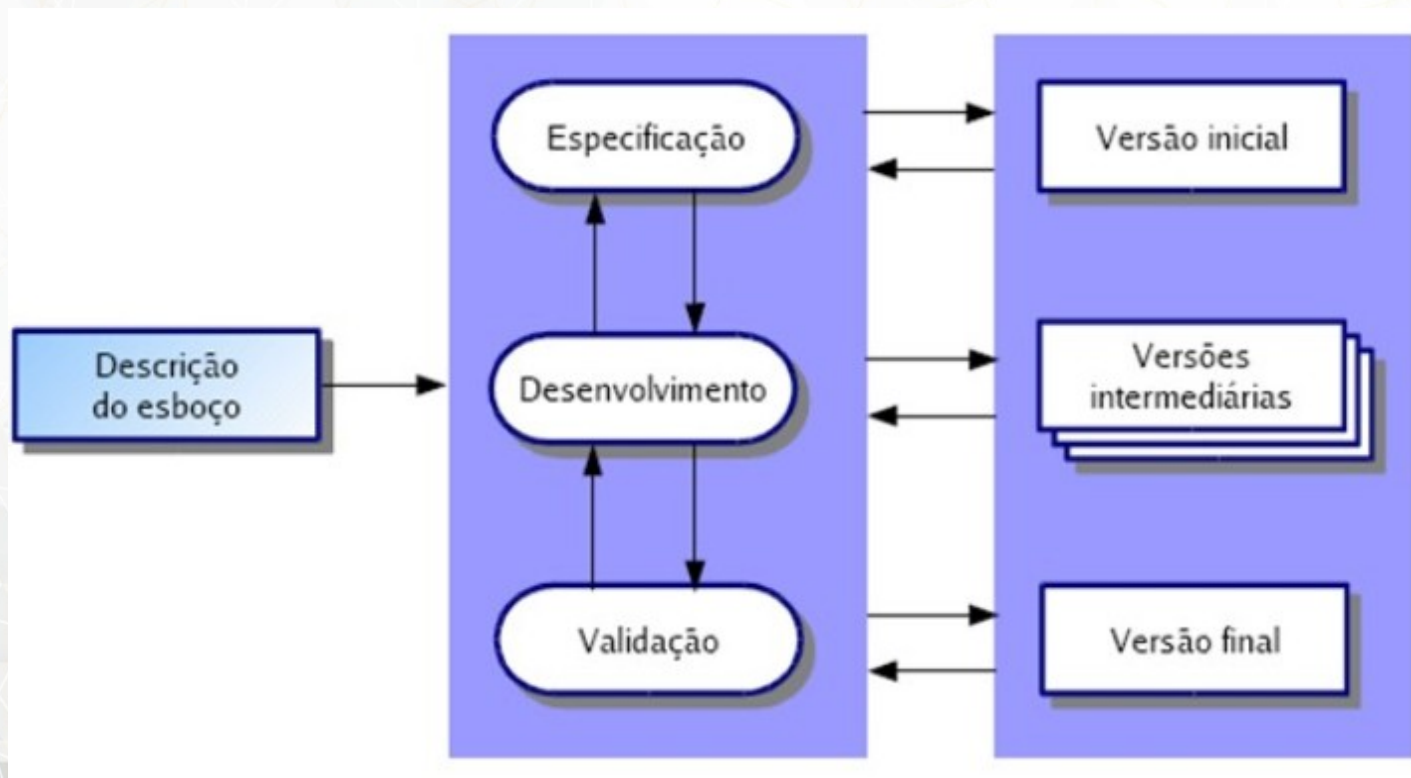
Etapas comuns do desenvolvimento

- Especificação de software, em que clientes e engenheiros definem/especificam o software a ser produzido, requisitos e restrições
- Desenvolvimento de software, em que o software é projetado e implementado (design e codificação)
- Validação de software, em que o software é analisado para garantir que é o que o cliente deseja
- Evolução de software, em que o software é modificado para refletir mudança de requisitos do cliente e/ou do mercado

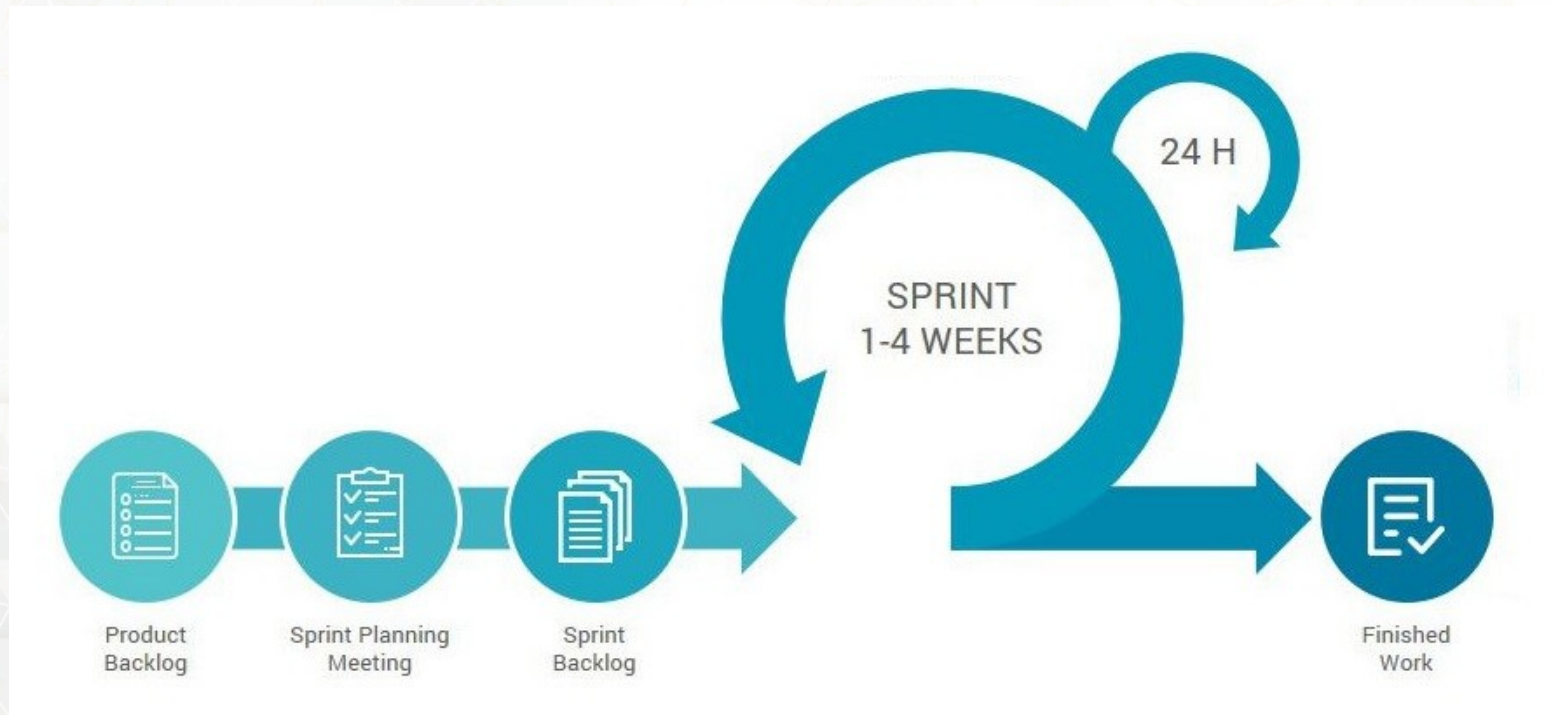
Modelo em cascata



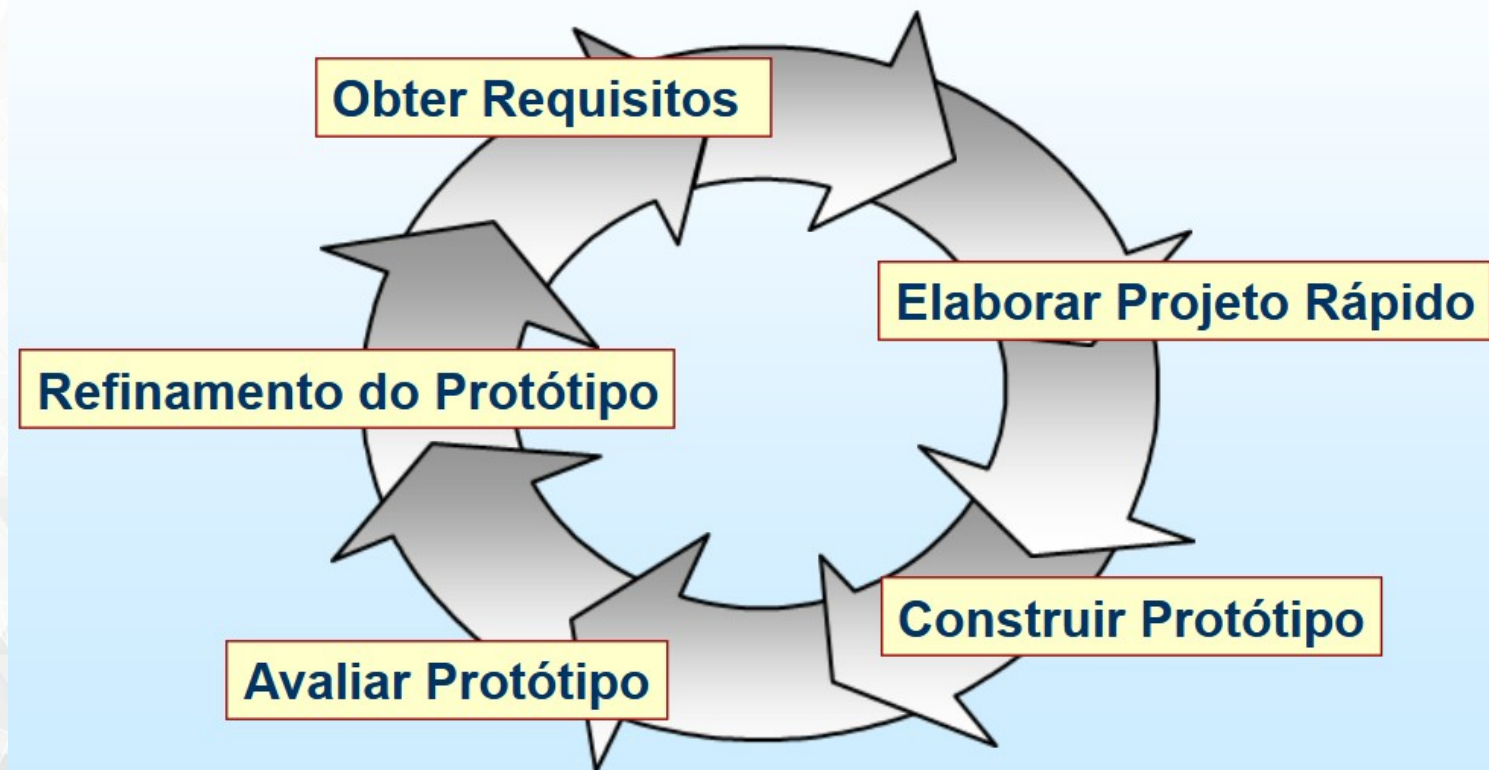
Modelo incremental

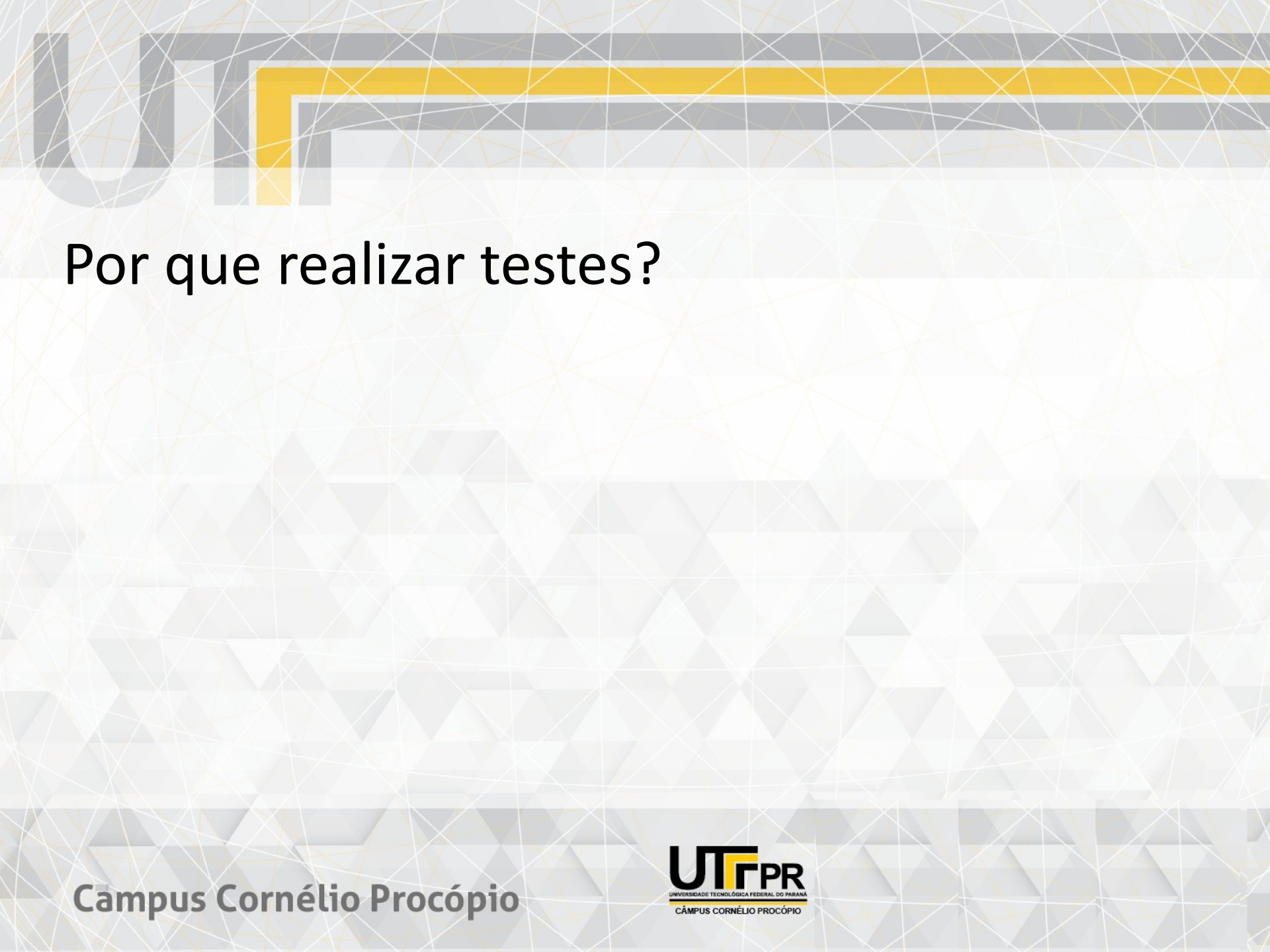


SCRUM



Prototipação





Por que realizar testes?

Por que realizar testes?

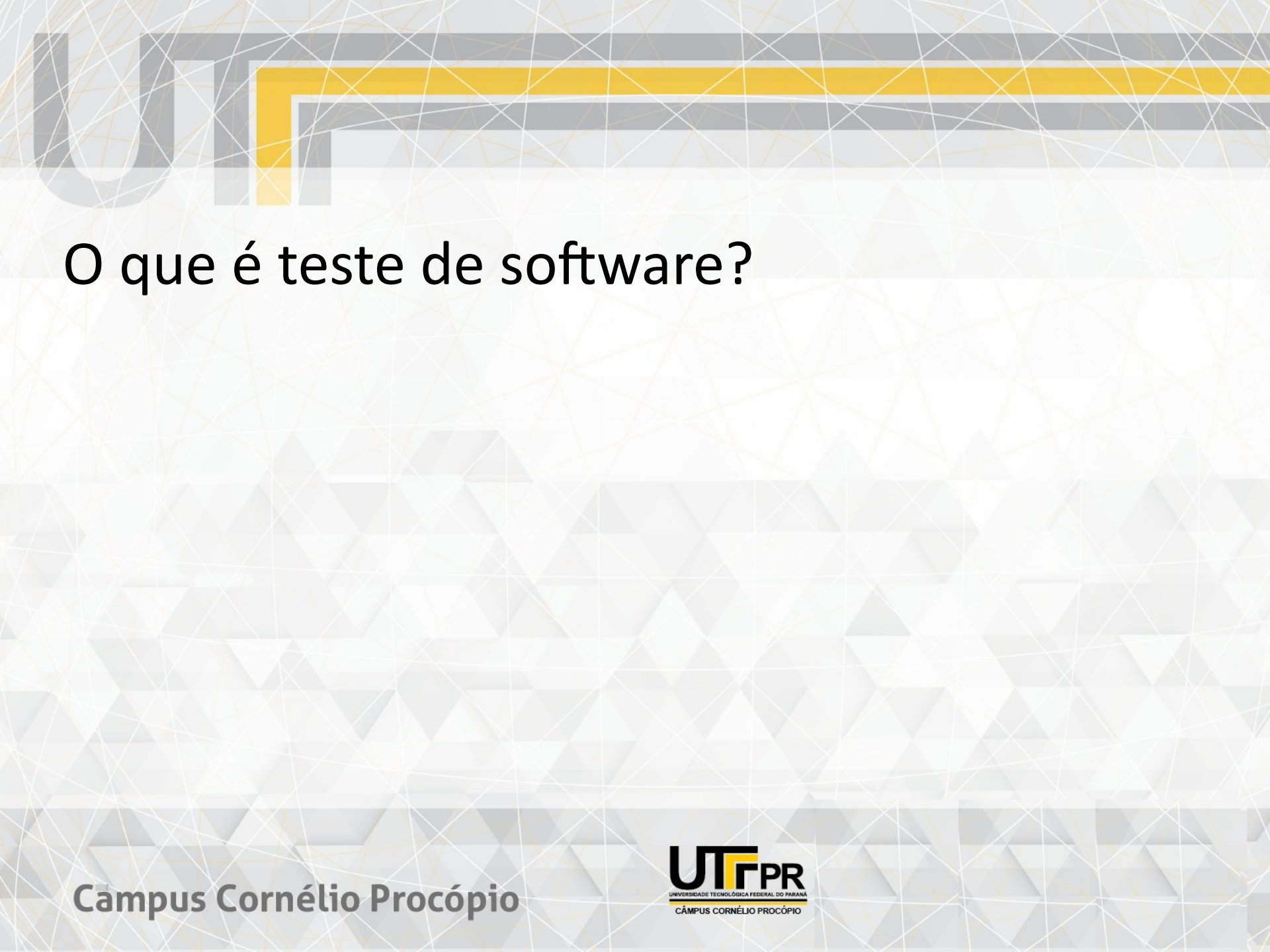
- Demanda por produtos de software de alta qualidade
- Quase não existem módulos livres de defeitos durante o desenvolvimento. Após a sua liberação, em torno de 40% podem estar livres de defeitos (SHULL et al., 2002)
- É quase improvável liberar um produto de software livre de defeitos (BOEHM e BASILI, 2001)
- Quanto mais tarde uma falha é revelada ou um defeito é identificado, maior será o custo na correção

Por que realizar testes?

- Therac-25
 - Máquina de radioterapia controlada por computador
 - Doses indevidas de radiação
 - Pelo menos 5 mortes entre 1985 e 1987

Por que realizar testes?

- Ariane 5
 - Foguete lançador de satélites
 - Destruição do foguete após o lançamento no ano de 1996
 - Prejuízo de 370 milhões de dólares



O que é teste de software?

O que é teste de software?

- Teste é o processo de executar um sistema ou componente sob condições específicas, observando e registrando os resultados, avaliando determinados aspectos do sistema ou componente (ISO/IEC/IEEE, 2010)

O que é teste de software?

- Teste é uma atividade que tem o objetivo de avaliar um atributo de um programa ou sistema. Teste é uma medida de qualidade de software (HETZEL, 1988)
- Teste é um processo de engenharia concorrente ao processo de ciclo de vida do software, que faz uso e mantém artefatos de teste usados para medir e melhorar a qualidade do produto de software em teste (CRAIG e JASKIEL, 2002)

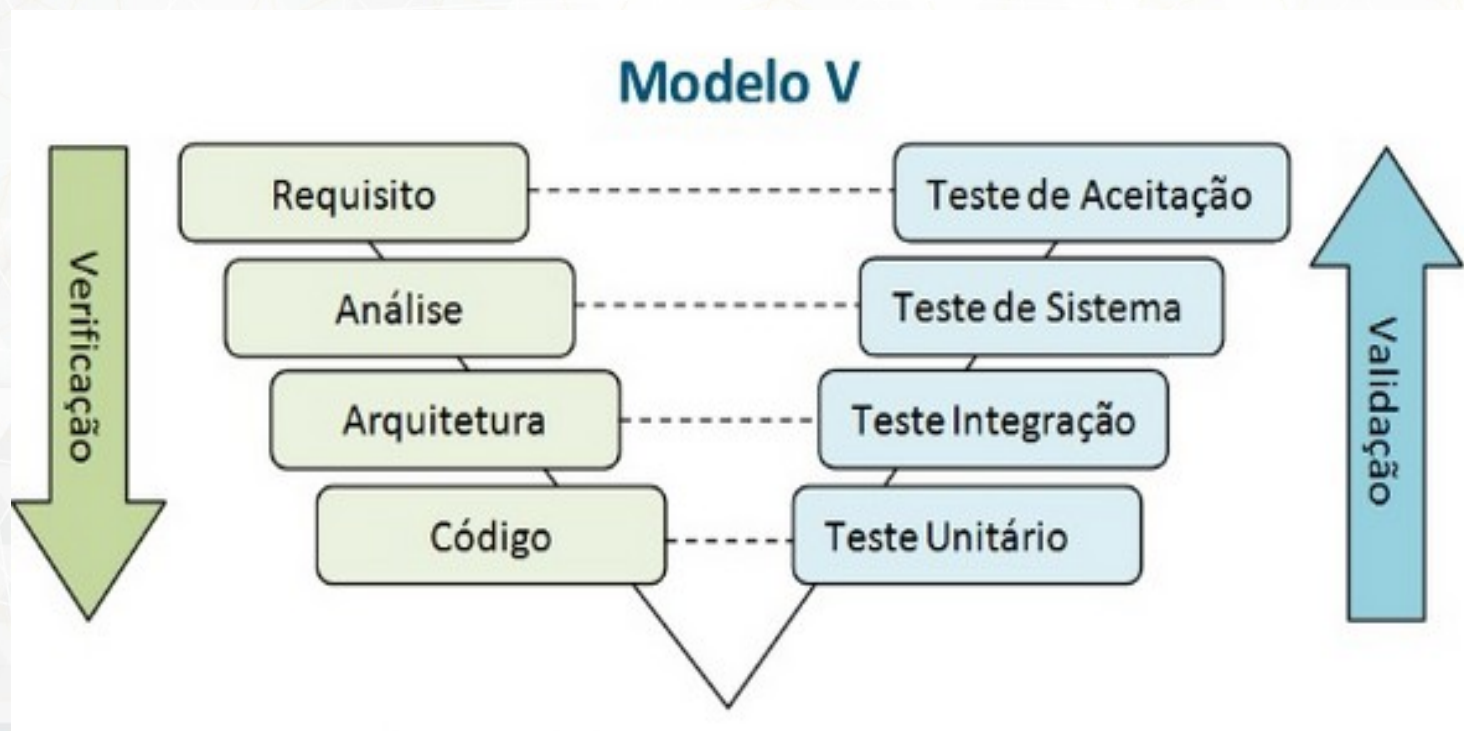
O que é teste de software?

- Teste é o processo de executar um programa ou sistema com a intenção de encontrar **erros** (MYERS, 1979)
- Teste pode ser usado apenas para mostrar a presença de defeitos, mas nunca sua ausência (DIJKSTRA, 1970)



Validação e Verificação

Validação e Verificação



Validação

- Estamos construindo o sistema correto? (BOEHM, 1981)
- Processo de avaliar o software ao final de seu processo de desenvolvimento para garantir que está de acordo com o uso pretendido (AMMANN e OFFUTT, 2008; ISO/IEC/IEEE, 2010)



Verificação

- Estamos construindo corretamente o sistema? (BOEHM, 1981)
- Processo para determinar se os produtos de uma determinada fase de desenvolvimento satisfazem os requisitos estabelecidos na fase anterior (AMMANN e OFFUTT, 2008; ISO/IEC/IEEE, 2010)

Terminologia

- A ISO/IEC/IEEE 24765:2010(E) *Systems and Software Engineering – Vocabulary* (ISO/IEC/IEEE, 2010) define os seguintes termos:
 - Engano (*Mistake*) – ação humana que produz um resultado incorreto. Exemplo: cometer um erro de digitação durante a implementação do software

Terminologia

- A ISO/IEC/IEEE 24765:2010(E) *Systems and Software Engineering – Vocabulary* (ISO/IEC/IEEE, 2010) define os seguintes termos:
 - Defeito (*Fault*) – um passo, processo, ou definição de dados incorreta em um produto de software. No uso comum, os termos “erro”, “bug”, e “defeito” são usados para expressar esse significado. Exemplo: implementação/codificação incorreta no software
 - O termo *defect* pode ser encontrado na literatura

Terminologia

- A ISO/IEC/IEEE 24765:2010(E) *Systems and Software Engineering – Vocabulary* (ISO/IEC/IEEE, 2010) define os seguintes termos:
 - Erro (*Error*) – diferença entre o valor computado, observado ou medido e o valor teoricamente correto de acordo com a especificação. Exemplo: divergência no estado do software (variável armazenando valor incorreto)

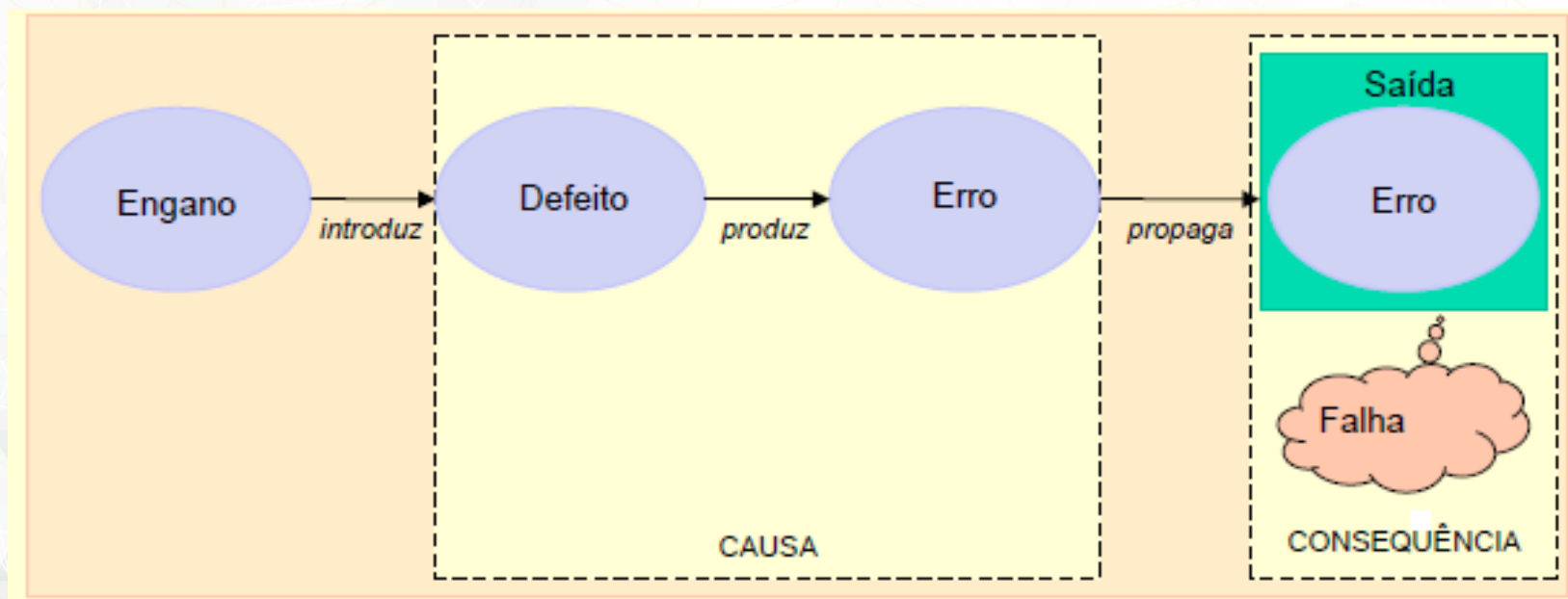
Terminologia

- A ISO/IEC/IEEE 24765:2010(E) *Systems and Software Engineering – Vocabulary* (ISO/IEC/IEEE, 2010) define os seguintes termos:
 - Falha (*Failure*) – inabilidade do sistema ou componente realizar a função requerida, considerando as questões de desempenho exigidas. Exemplo: resultado incorreto de uma funcionalidade do software

Terminologia

- Definição para os termos (AMMANN e OFFUTT, 2017):
 - *Fault* – a static defect in the software
 - *Error* – an incorrect internal state that is the manifestation of some fault
 - *Failure* – external, incorrect behavior with respect to the requirements or other description of the expected behavioral

Terminologia



Terminologia

- O domínio de entrada de um programa ou software P , denotado por $D(P)$, é o conjunto de todos os possíveis valores que podem ser utilizados na execução de P
- Um dado de teste para P é um elemento do domínio de entrada
- Uma ou mais saídas $S(P)$ são esperadas para $D(P)$

Terminologia

- Conjunto de casos de teste T
 - Caso de teste
 - Descrição de pré-condições, entrada(s) (dados de teste) e saída(s) esperada(s) para testar um P ou parte(s) de um P
 - Determinados autores incluem procedimento de execução
 - Dependendo do contexto, título e descrição podem ser inseridos
 - Todos os casos de teste formam o conjunto de teste
 - Somente um caso de teste normalmente não é suficiente

Terminologia

- Teste de software: definir um conjunto de casos de teste T para executar o programa P com T e verificar o resultado
 - Se o resultado obtido coincidir com o resultado esperado, nenhuma falha foi revelada
 - Se o resultado obtido for diferente do resultado esperado, uma falha foi revelada

Exemplo

```
2  /*Contagem da quantidade de elementos zero de um vetor*/
3  public int countZero(int[] vector) {
4      int count = 0;
5      if (vector.length > 0) {
6          for (int index = 1; index < vector.length; index++) {
7              if (vector[index] == 0) {
8                  count++;
9              }
10         }
11     }
12     return count;
13 }
```

Exemplo

- Verificar se o código possui defeito
- Especificar casos de teste para o código
 - Um caso em que o defeito não é executado
 - Um caso em que o defeito é executado, mas a falha não é revelada
 - Um caso em que o defeito é executado e a falha é revelada

Exemplo

- Especificar casos de teste para o código
 - Um caso em que o defeito é executado, mas a falha não é revelada

Caso de teste 1

Testar método de contagem de elementos zero de um vetor de inteiros

Entrada: vector = {1, 0, 0, 4}

Pré-condição: não há

Saída: count = 2

Procedimento de execução: -

Execução do caso de teste (comparação entre saída esperada e saída obtida)

- Saída obtida: count = 2

Exemplo

- Especificar casos de teste para o código
 - Um caso em que o defeito é executado e a falha é revelada

Caso de teste 2

Testar método de contagem de elementos zero de um vetor de inteiros

Entrada: vector = {0, 1, 0, 4}

Pré-condição: não há

Saída: count = 2

Procedimento de execução: -

Execução do caso de teste (comparação entre saída esperada e saída obtida)

- Saída obtida: **count = 1**

Exemplo

- Defeito: linha 5 (for (int index = 0; ...))
- Especificar casos de teste para o código
 - Um caso em que o defeito não é executado

Caso de teste 3

Testar método de contagem de elementos zero de um vetor de inteiros

Entrada: vector = { }

Pré-condição: não há

Saída: count = 0

Procedimento de execução: -

Execução do caso de teste (comparação entre saída esperada e saída obtida)

- Saída obtida: count = 0

Exercício 1

```
4  /*Contagem da quantidade de elementos positivos de um vetor*/
5  public int countPositive(int[] vector) {
6      int count = 0;
7      if (vector.length > 0) {
8          for (int index = 0; index < vector.length; index++) {
9              if (vector[index] >= 0) {
10                 count++;
11             }
12         }
13     }
14     return count;
15 }
```


Exercício 1

- Verificar se o código possui defeito
- Especificar casos de teste para o código
 - Um caso em que o defeito não é executado
 - Um caso em que o defeito é executado, mas a falha não é revelada
 - Um caso em que o defeito é executado e a falha é revelada

Exercício 2

- Implementar um método para retornar o índice do último elemento igual a zero de um vetor de inteiros. Não havendo elemento igual a zero, retornar -1.

Exercício 2

```
4  /*Verificação do índice do último elemento igual a zero.  
5   Não havendo elemento igual a zero, retornar -1*/  
6  public int findLastZero(int[] vector) {  
7      if (vector.length > 0) {  
8          for (int index = 0; index < vector.length; index++) {  
9              if (vector[index] == 0) {  
10                 return index;  
11             }  
12         }  
13     }  
14     return -1;  
15 }
```

Exercício 2

- Verificar se o código possui defeito
- Especificar casos de teste para o código
 - Um caso em que o defeito não é executado
 - Um caso em que o defeito é executado, mas a falha não é revelada
 - Um caso em que o defeito é executado e a falha é revelada

Dúvidas

Referências

- AMMANN, P.; OFFUT, J. Introduction to software testing. Cambridge University Press, 2008.
- BEIZER, B. Software testing techniques. 2nd ed. New York: Van Nostrand Reinhold Company, 1990.
- BINDER, R. V. Testing object-oriented systems: Models, patterns, and tools, v. 1. Addison Wesley Longman, Inc., 1999.
- BOEHM, B.; BASILI, V. R. Software defect reduction top 10 list. Computer, v. 34, n. 1, p. 135–137, 2001.
- BOEHM, B. W. Software engineering economics. 1st ed. Upper Saddle River, NJ, USA: Prentice Hall PTR, 1981.
- BOEHM, B. W. Industrial software metrics top 10 list. IEEE Software, v. 4, n. 5, p. 84–85, 1987.
- CHOW, T. S. Testing software design modelled by finite-state machines. IEEE Transactions on Software Engineering, v. 4, n. 3, p. 178–187, 1978.
- COPELAND, L. A practitioner's guide to software test design. Artech House Publishers, 2004.
- CRAIG, R. D.; JASKIEL, S. P. Systematic software testing. Artech House Publishers, 2002.
- DELAMARO, M. E.; MALDONADO, J. C.; JINO, M. Introdução ao Teste de Software. Rio de Janeiro: Elsevier, 2007.
- DIJKSTRA, E. W. Notes on structured programming. Relatório Técnico 70-WSK-03, Technological University of Eindhoven, The Netherlands, available at: <http://www.cs.utexas.edu/users/EWD/ewd02xx/EWD249.PDF>. Accessed on: 10-27-2007., 1970.

Referências

- HETZEL, B. The complete guide to software testing. 2nd ed. Wellesley, MA, USA: QED Information Sciences, Inc., 1988.
- IBM Orthogonal defect classification v 5.2 for software design and code. Página Web, disponível em: http://researcher.watson.ibm.com/researcher/view_project.php?id=480. 2013, Acesso em: 02/02/2014.
- ISO/IEC Quality characteristics and guidelines for their use. Padrão ISO/IEC 9126, ISO/IEC, 1991. ISO/IEC/IEEE Systems and software engineering – vocabulary. 2010.
- MALDONADO, J. C.; BARBOSA, E. F.; VINCENZI, A. M. R.; DELAMARO, M. E.; SOUZA, S. R. S.; JINO, M. Introdução ao teste de software. Relatório Técnico 65 – Versão 2004-01, Instituto de Ciências Matemáticas e de Computação – ICMC-USP, disponível on-line: http://www.icmc.usp.br/CMS/Arquivos/arquivos_enviados/BIBLIOTECA_113_ND_65.pdf,2004.
- MYERS, G. J. The art of software testing. Wiley, New York, 1979.
- PEZZÉ, M.; YOUNG, M. Software testing and analysis: Process, principles and techniques. John Wiley & Sons, 2007.
- ROPER, M. Software testing. McGrall Hill, 1994.
- SHULL, F.; BASILI, V.; BOEHM, B.; BROWN, A. W.; COSTA, P.; LINDVALL, M.; PORT, D.; RUS I.; TESORIERO, R.; ZELKOWITZ, M. What we have learned about fighting defects. In: VIII International Symposium on Software Metrics -METRICS'02, Washington, DC, USA: IEEE Computer Society, 2002, p. 249–258.